

1 Tối ưu hoá và siêu tham số (Optimization and Hyperparameters)

PHỤ THUỘC C/12/02-lam-cho-training-chay-duoc (gradient phải lành mạnh trước khi bàn tới bước đi); A/01-toan-toi-thieu (số điều kiện, hội tụ của gradient descent).

TÓM TẮT

Mục này đọc bảy *optimizer* quen thuộc như một chuỗi vá lỗi liên tiếp chứ không như một menu lựa chọn, rồi chỉ ra siêu tham số nào thật sự quyết định kết quả. Đọc xong bạn nhìn một đường cong học và đoán được đang sai ở đâu, đặt được *learning rate* và lịch của nó có căn cứ thay vì mò, và giải thích được vì sao *mixed precision* cần *loss scaling*. Nếu chỉ nhớ một điều: optimizer chỉ định hình lại bước đi, nó không cứu được một learning rate sai — nên hãy tune learning rate trước, batch size sau, và chọn optimizer cuối cùng.

KIẾN THỨC CẦN CÓ TRƯỚC

- **mô hình (model)** — một hàm có tham số, nhận đầu vào và trả về dự đoán; huấn luyện là đi tìm bộ tham số làm dự đoán sát nhất.
- **gradient** — vectơ đạo hàm riêng của hàm mất mát theo từng tham số; nó chỉ hướng làm mất mát TĂNG nhanh nhất, nên mọi thuật toán ở đây đi ngược lại nó.
- **learning rate** — độ dài bước mà thuật toán tối ưu đi theo hướng ngược gradient ở mỗi lần cập nhật; quá lớn thì nghiệm bật ra ngoài, quá nhỏ thì hội tụ chậm tới mức vô dụng.
- **optimizer (thuật toán tối ưu)** — quy tắc biến gradient thành bước cập nhật thực tế; toàn bộ mục này là lịch sử bảy quy tắc như vậy và lý do từng cái ra đời.
- **siêu tham số (hyperparameter)** — con số do người đặt trước khi huấn luyện chứ không được học từ dữ liệu — learning rate, batch size, cường độ weight decay.
- **bước và epoch (step / epoch)** — một “bước” là một lần cập nhật tham số dựa trên một lô dữ liệu; một “epoch” là khi mô hình đã đi hết toàn bộ tập huấn luyện một lượt.
- **weight decay** — cơ chế kéo các trọng số về gần 0 sau mỗi bước, nhằm hạn chế mô hình bám quá sát dữ liệu huấn luyện; cách nó được cài đặt là toàn bộ khác biệt giữa Adam và AdamW.
- **fp32, fp16, bf16** — ba định dạng số thực với số bit khác nhau. Điều quan trọng không phải “bao nhiêu bit” mà là bit đó chia thế nào giữa *mũ* (quyết định dải giá trị biểu diễn được) và *định trị* (quyết định độ chính xác).

A. Đặt vấn đề

Giả sử gradient đã tính đúng và không còn chết hay nổ. Câu hỏi còn lại nghe rất đơn giản: biết hướng dốc rồi thì đi bao xa? Nhưng nó khó vì ba lý do chồng lên nhau. Thứ nhất, mất mát bị điều kiện xấu — có hướng dốc gấp trăm lần hướng khác, nên một bước dài theo hướng này là bước ngắn vô vọng theo hướng kia. Thứ hai, gradient bạn có không phải gradient thật mà là ước lượng từ một minibatch, nên nó nhiễu. Thứ ba, mọi thứ phi lỗi, nên không có lý thuyết nào nói cho bạn bước đi tối ưu.

Toàn bộ lịch sử *optimizer* trong deep learning là câu trả lời cho ba khó khăn ấy, và nó là ví dụ tốt nhất trong tài liệu này về việc *một danh sách thực ra là một mạch lập luận*. Bảy cái tên dưới đây thường được dạy như bảy lựa chọn ngang hàng trong một menu. Chúng không ngang hàng: mỗi cái ra đời để sửa một khiếm khuyết cụ thể của cái trước, và mỗi cái tạo ra một khiếm khuyết mới cho cái sau. Nếu bạn đọc được mạch đó thì bạn không cần nhớ công thức, vì bạn suy lại được chúng; nếu bạn chỉ nhớ danh sách thì bạn sẽ chọn optimizer theo thói quen và không bao giờ biết mình đang trả giá gì.

TRỰC GIÁC

Momentum thường được kể bằng ẩn dụ hòn bi lăn xuống dốc, nhưng ẩn dụ đó không nói cho bạn biết *vì sao* nó hiệu quả. Cách nhìn đúng hơn với người đã lọc tín hiệu IMU: chuỗi gradient theo thời gian là một tín hiệu, và momentum là một bộ lọc thông thấp một cực đặt lên tín hiệu đó. Trong một rãnh hẹp, thành phần vuông góc với rãnh đổi dấu mỗi bước — đó là thành phần tần số cao, và bộ lọc dập nó. Thành phần dọc theo rãnh giữ nguyên dấu qua nhiều bước — đó là thành phần một chiều, và bộ lọc để nó cộng dồn tới hệ số $1/(1 - \mu)$. Momentum không “thêm quán tính”; nó tách tín hiệu khỏi dao động, đúng như khi bạn lọc bias con quay ra khỏi rung động khung máy.

B. Bảy bước của một mạch lập luận duy nhất (SGD → AdamW)

Trước khi vào mạch, hãy cố định một con số để có cảm giác về mức độ nghiêm trọng. Với một mặt bậc hai có số điều kiện κ , gradient descent hội tụ với hệ số $\left(\frac{\kappa-1}{\kappa+1}\right)$ mỗi bước. Với $\kappa = 1000$, hệ số đó là 0,998, nên cần khoảng 350 bước chỉ để giảm sai số một nửa. Mọi thứ trong mạch dưới đây là những cách khác nhau để không phải trả cái giá đó.

→ **tiền đề** A/01 — số điều kiện và hệ số hội tụ của gradient descent trên mặt bậc hai là kết quả chuẩn của giải tích số

I Mạch 1 — tiến hoá của optimizer

- 1 Vấn đề** gradient descent ngẫu nhiên đi zigzag trong rãnh hẹp và bò rất chậm theo hướng thoải.

Giải pháp **hạ gradient ngẫu nhiên** (*stochastic gradient descent, SGD*), $\theta \leftarrow \theta - \eta g$, là khởi điểm và cũng là chuẩn so sánh; ưu điểm thật của nó là nhiều *minibatch* đóng vai trò **chính quy hoá ngầm** (*implicit regularization*).

Giá phải trả bước học η phải nhỏ hơn nghịch đảo độ cong lớn nhất để không phân kỳ, nên nó tự động quá nhỏ cho mọi hướng còn lại.

Vấn đề mới tốc độ bị khoá vào κ , và một giá trị η duy nhất phải phục vụ mọi tham số trong mạng.
- 2 Vấn đề** dao động vuông góc với rãnh làm phí gần hết công.

Giải pháp **momentum**: $v \leftarrow \mu v + g$, $\theta \leftarrow \theta - \eta v$. Thành phần đổi dấu triệt tiêu trong tổng trượt, thành phần cùng dấu cộng dồn tới hệ số $1/(1 - \mu)$, tức gấp 10 lần khi $\mu = 0,9$.

Hệ quả tốc độ hội tụ chuyển từ phụ thuộc κ sang phụ thuộc $\sqrt{\kappa}$, một cải thiện có chứng minh cho trường hợp bậc hai.

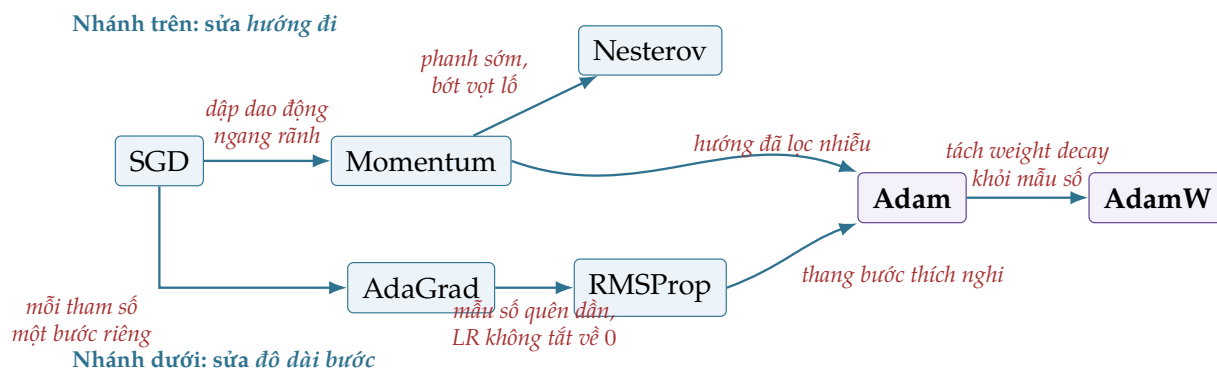
Vấn đề mới momentum đã tích lũy không dừng lại được ngay: nghiệm bị vọt lố qua điểm cực tiểu, và trong một rãnh cong thì vọt lố nghĩa là văng ra thành.
- 3 Vấn đề** vọt lố vì gradient được đo ở nơi ta *đang* đứng chứ không phải nơi ta *sắp* tới.

Giải pháp **Nesterov**: tính gradient tại điểm nhìn trước $\theta - \eta \mu v$. Hiệu giữa gradient ở điểm nhìn trước và gradient tại chỗ là một xấp xỉ của số hạng bậc hai. Nó hoạt động như lực hãm tỉ lệ với tốc độ thay đổi của gradient, tức phanh trước khi đâm vào thành chứ không sau.

Giá phải trả thêm một lần bookkeeping, thường được cài bằng phép đổi biến nên chi phí thực tế bằng không.

Vấn đề mới vẫn còn nguyên vấn đề gốc: một η chung cho tất cả tham số.

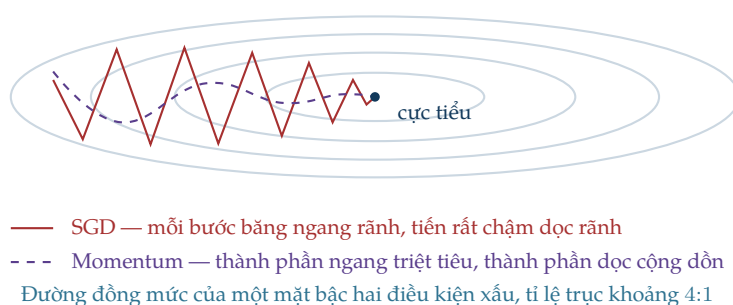
- 4 Vấn đề** các tham số có độ lớn gradient lệch nhau hàng bậc, nên không tồn tại một η đúng cho tất cả.
- Giải pháp** *AdaGrad*: tích lũy $G \leftarrow G + g^2$ theo từng tọa độ và bước theo $\eta g / (\sqrt{G} + \varepsilon)$. Tham số hiếm được cập nhật nhận bước lớn, tham số hay được cập nhật nhận bước nhỏ; ý tưởng sinh ra từ các bài toán lỗi thừa và rất hợp lý ở đó.
- Vấn đề mới** G là một tổng đơn điệu tăng, nên bước học hiệu dụng tắt dần về 0. Trong một bài toán lỗi thì đó là tính năng; trong một mạng sâu chạy hàng trăm nghìn bước thì đó là cái chết: mô hình ngừng học trong khi loss còn cao.
- 5 Vấn đề** mẫu số của AdaGrad nhớ quá lâu.
- Giải pháp** *RMSProp*: thay tổng bằng trung bình trượt phân rã, $G \leftarrow \beta G + (1 - \beta)g^2$. Bộ nhớ trở thành hữu hạn với chân trời khoảng $1/(1 - \beta)$ bước, tức chừng 1000 bước khi $\beta = 0,999$, nên bước học hiệu dụng không tắt.
- Vấn đề mới** hướng đi vẫn là gradient thô nên vẫn nhiễu; và trung bình trượt khởi tạo từ 0 nên trong những bước đầu G bị đánh giá thấp, mẫu số quá nhỏ, bước đi quá dài đúng vào lúc mô hình dễ tổn thương nhất.
- 6 Vấn đề** cần cả hai: hướng đã lọc nhiễu và thang bước riêng cho từng tham số.
- Giải pháp** *Adam* [1] giữ hai trung bình trượt: m cho gradient và v cho bình phương gradient. Nó hiệu chỉnh độ chệch bằng $\hat{m} = m/(1 - \beta_1^t)$ và $\hat{v} = v/(1 - \beta_2^t)$, rồi bước theo $\eta \hat{m} / (\sqrt{\hat{v}} + \varepsilon)$. Hiệu chỉnh độ chệch không phải chi tiết trang trí. Ở bước $t = 1$ với $\beta_1 = 0,9$ và $\beta_2 = 0,999$, ta có $m = 0,1g$ và $\sqrt{\hat{v}} = 0,0316|g|$. Bước chưa hiệu chỉnh vì thế dài gấp khoảng 3,2 lần bước mong muốn.
- Giá phải trả** hai trạng thái phụ cho mỗi tham số: với mô hình 100 triệu tham số ở fp32, tổng bộ nhớ là $4P$ cho trọng số, $4P$ cho gradient và $8P$ cho trạng thái, tức 1,6 GB trước khi tính một byte kích hoạt nào.
- Vấn đề mới** hình phạt L2 cộng vào gradient bị chia cho $\sqrt{\hat{v}}$ giống hết phần còn lại, nên tham số nào có lịch sử gradient lớn thì bị suy giảm ít đi — nó không còn là weight decay theo nghĩa ban đầu nữa.
- 7 Vấn đề** weight decay trong Adam không phải weight decay.
- Giải pháp** *AdamW* [2] tách hẳn hai thứ: $\theta \leftarrow \theta - \eta(\hat{m}/(\sqrt{\hat{v}} + \varepsilon) + \lambda\theta)$, tức phần suy giảm đi thẳng vào trọng số, không qua mẫu số thích nghi.
- Hệ quả** suy giảm trở lại thành một phép co đều, độc lập với độ lớn gradient, và đây là lý do AdamW là mặc định thực tế cho transformer lẫn cho các recipe CNN hiện đại.
- Giá phải trả** λ không còn so sánh được với λ của Adam cũ, nên mọi giá trị chép từ code cũ đều sai thang.
- Vấn đề mới** phần suy giảm nay tỉ lệ với η , nên nó tương tác với lịch học: giảm η theo cosine thì cũng đang giảm dần cường độ chính quy hoá, một hiệu ứng ít người nói ra.



Hình 1. Tiến hoá của optimizer không phải một chuỗi thẳng mà là hai nhánh độc lập rồi hợp lại. Nhánh trên sửa *hướng* của bước đi bằng cách lọc nhiễu theo thời gian; nhánh dưới sửa *độ dài* bước riêng cho từng tham số. Adam là chỗ hai nhánh gặp nhau, và AdamW là bản vá cho khiếm khuyết mà chính phép hợp nhất đó sinh ra. Mỗi mũi tên ghi đúng khiếm khuyết mà bước sau gỡ được.

Hình 1 nên được nhìn trước khi đọc mạch bảy bước, vì nó cho thấy ngay điều mà đọc tuần tự khó thấy: Adam không phải “bước tiến thứ sáu” mà là phép hợp nhất hai dòng giải quyết hai vấn đề khác nhau, và đó cũng là lý do nó thừa hưởng nhược điểm của cả hai.

Hai nhận xét khép lại mạch. Thứ nhất, hãy để ý rằng không bước nào trong bảy bước trên giải quyết vấn đề phi lỗi; tất cả đều xử lý điều kiện xấu và nhiễu. Optimizer không tìm cực tiểu toàn cục và không hứa điều đó. Thứ hai, cả họ thích nghi có một cái giá chung ít được nêu: chúng *che giấu* một learning rate sai. Adam với η lệch một bậc vẫn cho đường cong loss trông chấp nhận được, trong khi SGD với sai lệch tương tự thì phân kỳ ngay và bạn biết mình sai. Đây là lý do đáng giữ SGD với momentum làm phép thử chẩn đoán, ngay cả khi bản chạy cuối dùng AdamW.



Hình 2. Vì sao momentum thắng trong rãnh hẹp. Thành phần vuông góc với rãnh đổi dấu mỗi bước nên triệt tiêu trong tổng trượt; thành phần dọc rãnh giữ nguyên dấu nên cộng dồn. Tỉ lệ trục trong hình chỉ khoảng 4:1 cho dễ nhìn; trong mạng thật số điều kiện lớn hơn hai tới ba bậc, nên hiệu ứng mạnh hơn nhiều.

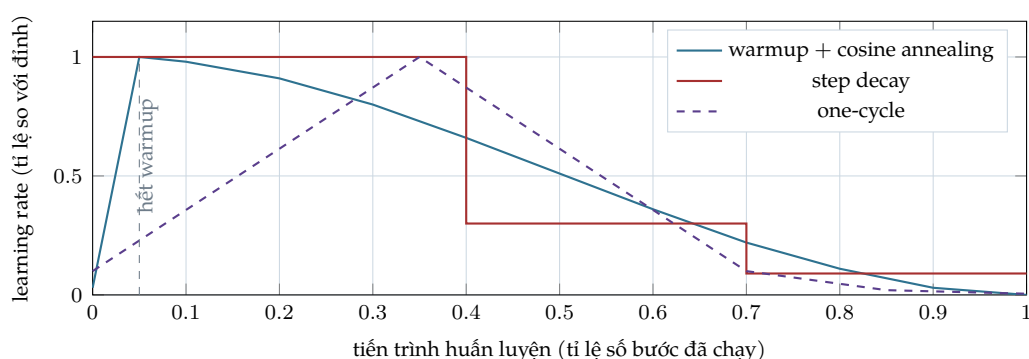
Hình 2 là toàn bộ trực giác hình học của bước hai trong mạch, và cũng cho thấy vì sao Nesterov cần thiết: đường đứt nét vẫn vọt qua tâm ở lần tiếp cận đầu tiên nếu μ đủ lớn.

C. Learning rate là siêu tham số quan trọng nhất, không có đối thủ

Nếu chỉ được tune một thứ, tune learning rate. Điều này nghe như khẩu hiệu, nhưng nó có cơ sở. η là đại lượng duy nhất nhân trực tiếp vào mọi bước cập nhật. Sai một bậc ở η đổi hẳn bản chất của quá trình; sai một bậc ở weight decay chỉ làm kết quả kém đi vài phần trăm. Kỹ năng đáng đầu tư nhất là nhận dạng *hình dạng* đường cong học, vì nó tiết kiệm hàng tuần chạy thử: loss tăng vọt hoặc thành NaN trong vài chục bước là η quá cao rõ ràng; loss tụt rất nhanh rồi phẳng ở mức cao và nhiễu mạnh là η hơi cao, hệ đang nảy quanh một lòng chảo mà không xuống được; loss giảm gần như tuyến tính và không có dấu hiệu bão hoà sau nhiều epoch là η quá thấp. Cách

re nhất để định vị là chạy một lượt quét: tăng η theo cấp số nhân trong vài trăm bước và vẽ loss theo η ; vùng dốc nhất trước điểm nổ là vùng đáng dùng.

warmup tăng η tuyến tính từ gần 0 lên giá trị đích, trong vài trăm tới vài nghìn bước. Thoạt nhìn nó là một thủ thuật tùy tiện. Thực ra nó có ít nhất hai lý do rõ ràng, và cả hai đều đã xuất hiện ở trên. Thứ nhất, mẫu số thích nghi của Adam ở những bước đầu được ước lượng từ rất ít mẫu, nên phương sai của $\hat{v}$ rất lớn; một toạ độ vô tình có $\hat{v}$ nhỏ sẽ nhận một bước khổng lồ và phá hỏng số ngay khi chưa có gì để cứu. Thứ hai, thống kê chạy của BatchNorm chưa hội tụ trong vài trăm bước đầu, nên hàm mà bạn đang tối ưu vẫn còn đang thay đổi dưới chân. Warmup mua thời gian cho cả hai bộ ước lượng đó. **Lịch learning rate** thì giải quyết một vấn đề khác: SGD với η cố định không hội tụ về một điểm mà tiến tới một phân bố dừng có phương sai tỉ lệ với η ; giảm η là thu nhỏ bán kính quả cầu mà nghiệm đang lang thang trong đó. Ba lịch phổ biến là step decay, cosine annealing [3] và one-cycle; khác biệt giữa chúng nhỏ hơn nhiều so với khác biệt giữa “có lịch” và “không có lịch”.



Hình 3. Ba lịch learning rate trên cùng một trục. Điều hình cho thấy mà lời văn khó nói: chúng khác nhau ở *chỗ* tiêu tiền, không ở tổng lượng. Step decay giữ nguyên bước dài rất lâu rồi cắt đột ngột. Cosine hạ liên tục nên tránh được cú sốc ở mỗi mốc cắt. One-cycle thì cố tình bắt đầu thấp và *tăng* lên đỉnh ở giữa, để đi qua vùng nhiễu nhanh hơn. Đoạn dốc đứng ở đầu đường cosine là warmup: nó tồn tại vì bộ ước lượng $\hat{v}$ của Adam và thống kê BatchNorm đều chưa đáng tin ở vài trăm bước đầu.

Hình 3 cũng cho thấy vì sao so sánh hai bài báo dùng hai lịch khác nhau là việc rất dễ sai: hai mô hình cùng số epoch có thể đã trải qua hai quá trình rất khác nhau.

D. Đổi batch size thì đổi gì theo (linear scaling rule)

quy tắc tỉ lệ tuyến tính (*linear scaling rule*) nói rằng nhân batch size lên k lần thì nhân learning rate lên k lần. Quy tắc này đã được kiểm chứng trên ImageNet với warmup đủ dài, tới batch 8192 [4]. Lập luận đơn giản: một bước với batch kB nên tiến xa bằng k bước với batch B , và trung bình của k gradient liên tiếp xấp xỉ gradient của batch gộp khi trọng số chưa đổi nhiều.

Nhưng nguyên nhân sâu hơn mới đáng nhớ, vì nó nói rằng đây không chỉ là chuyện tốc độ. Phương sai của gradient minibatch tỉ lệ nghịch với batch size, nên batch lớn cho gradient sạch hơn — và nhiễu gradient chính là một dạng chính quy hoá ngầm. Batch quá lớn không chỉ “khó tune hơn”: nó *lấy đi* một nguồn chính quy hoá, và các quan sát trên ImageNet cho thấy vượt một ngưỡng nhất định thì độ chính xác kiểm thử tụt dù loss huấn luyện vẫn tốt [5]. Smith và Le đưa ra một cách nhìn thống nhất: giảm η và tăng batch size là hai cách làm cùng một việc, đều là giảm mức nhiễu η/B [6]. Cách nhìn đó giải thích cả linh cảm “tăng batch thì phải tăng LR”, lẫn lý do quy tắc vỡ. Khi η đã lớn tới mức không tăng được nữa vì lý do ổn định, mỗi liên hệ đứt.

↪ **đối lập với** C/12/04 — nhiễu gradient vừa là thứ quy tắc tỉ lệ tuyến tính muốn bảo toàn, vừa là nguồn chính quy hoá ngầm mà batch lớn xoá mất

E. Theo dõi sức khoẻ huấn luyện (training health), vì loss là chỉ báo quá chậm

Đường cong loss cho biết *đã* có vấn đề, thường sau hàng nghìn bước. Ba phép đo dưới đây rẻ như nhau nhưng phát hiện sớm hơn nhiều, vì chúng đo trạng thái bên trong chứ không đo kết quả.

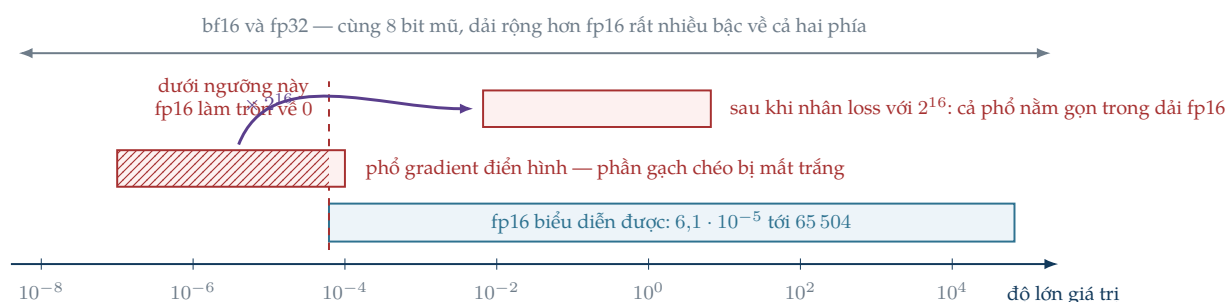
- **Tỉ lệ cập nhật trên trọng số** (*update-to-weight ratio*), theo từng tầng: $\|\eta \Delta \theta\| / \|\theta\|$.
 - Kinh nghiệm thực hành phổ biến: khoảng 10^{-3} là lành mạnh; trên 10^{-2} là mỗi bước làm đổi trọng số vài phần trăm, tức quá lớn; dưới 10^{-4} là tầng đó thực chất đang đứng yên.
 - Vì sao dùng nó thay cho chuẩn gradient thô: nó không thứ nguyên, nên so sánh được giữa các tầng có thang hoàn toàn khác nhau.
- **Phân bố kích hoạt theo chiều sâu** — vẽ độ lệch chuẩn của kích hoạt từng tầng. Co dần theo chiều sâu là vanishing đang hình thành; phình lên là exploding. Đây chính là đại lượng ρ^L của mục trước, quan sát trực tiếp.
- **Tỉ lệ ReLU chết** — phần trăm nơ-ron cho 0 trên toàn bộ một batch xác thực. Vài phần trăm là bình thường; ba mươi phần trăm ở một tầng nghĩa là η đã có lúc quá cao, kể cả khi hiện tại nó đã ổn.

Cả ba chỉ báo chỉ tốn vài dòng hook và nên được ghi lại mặc định, vì giá trị của chúng nằm ở chỗ chúng *đi trước* loss chứ không đi sau.

F. Mixed precision: chuyện của dải động, không phải của độ chính xác

Chuyển sang **fp16** làm training ra NaN, và nguyên nhân thường bị hiểu nhầm là “thiếu độ chính xác”. Thủ phạm thật là *dải động*. fp16 có 5 bit mũ và 10 bit định trị, nên số chuẩn hoá nhỏ nhất là khoảng $6,1 \cdot 10^{-5}$ và số lớn nhất là 65 504. Gradient trong mạng thị giác thường nằm quanh 10^{-7} tới 10^{-4} ; phần lớn phổ đó rơi xuống dưới ngưỡng chuẩn hoá của fp16 và bị làm tròn về 0. Mạng không nổ, nó *tê liệt*: một phần các tầng đơn giản là không nhận được tín hiệu nào.

Lời sửa của **loss scaling** [7] tinh tế ở chỗ nó không đụng vào định dạng số. Nhân loss với một hệ số S cỡ 2^{16} trước khi gọi backward; vì backward tuyến tính theo adjoint đầu vào, mọi gradient bị nhân đúng S lần và cả phổ dịch lên vùng biểu diễn được; trước khi optimizer bước, chia gradient lại cho S . Bản động của kỹ thuật này nhân S lên gấp đôi sau mỗi vài nghìn bước sạch và chia đôi ngay khi phát hiện inf hoặc NaN, bỏ luôn bước đó — tức là nó tự dò biên trên của dải động.



Hình 4. Vì sao fp16 làm huấn luyện ra NaN, và vì sao loss scaling chữa được. Trục là độ lớn giá trị theo thang log. Dải fp16 (nền xanh) không bao phủ phần dưới của phổ gradient điển hình, nên phân gạch chéo bị làm tròn về 0 — mạng không nổ mà *tê liệt*. Nhân loss với 2^{16} dịch toàn bộ phổ sang phải mà không đổi hình dạng của nó, đưa cả phổ vào vùng biểu diễn được; chia lại trước bước cập nhật thì không mất gì. bf16 giải bài toán theo hướng khác: nó giữ nguyên số bit mũ của fp32 nên dải đủ rộng từ đầu, đổi lại chỉ còn 7 bit định trị.

Hình 4 làm rõ một điểm mà hai chữ “mixed precision” che mất: vấn đề nằm ở *vị trí* của phổ gradient so với dải biểu diễn được, không nằm ở số chữ số có nghĩa. Đó cũng là lý do lời sửa là một phép nhân vô hướng chứ không phải một thuật toán mới.

bf16 giải bài toán theo hướng khác: giữ 8 bit mũ như fp32 nên dải động y hệt fp32 và không cần loss scaling, đổi lại chỉ còn 7 bit định trị.

Chính 7 bit định trị đó dẫn tới điểm kỹ thuật đáng nhớ nhất của cả mục. Độ chính xác tương đối của bf16 là khoảng $2^{-8} \approx 4 \cdot 10^{-3}$. Nhưng ta vừa nói tỉ lệ cập nhật trên trọng số lành mạnh là khoảng 10^{-3} . Nghĩa là bước cập nhật *nhỏ hơn một đơn vị cuối cùng của trọng số*: nếu bạn cộng nó vào một trọng số lưu ở bf16, phép cộng làm tròn về đúng giá trị cũ và bước đi biến mất hoàn toàn. Đây là lý do mọi cài đặt mixed precision nghiêm túc giữ một bản *master weights* ở fp32 cho optimizer, dù forward và backward chạy ở nửa độ chính xác; và cũng là lý do bảng kế toán bộ nhớ không giảm một nửa như người ta hy vọng.

→ cùng nguyên lý với D/20 — lượng tử hoá lúc suy luận hỏi đúng câu này: bit chia thế nào giữa mũ và định trị.

Cuối cùng, **cắt ngưỡng gradient** (*gradient clipping*) theo chuẩn toàn cục là bắt buộc với RNN và thường cần với transformer: nó chặn bước đi khi mặt mất mát có vách dựng đứng, nơi gradient tại một điểm không còn nói gì đúng về hàm ở khoảng cách một bước.

G. Giới hạn và trường hợp hỏng

Linear scaling rule vỡ ở batch lớn, và nó vỡ theo kiểu khó phát hiện: loss huấn luyện vẫn đẹp, chỉ có độ chính xác kiểm thử tụt. Nếu bạn chỉ theo dõi loss huấn luyện, bạn sẽ kết luận rằng batch lớn hoạt động tốt.

Loss scaling che giấu NaN thật. Bộ chia tỉ lệ động bỏ qua mọi bước có tràn số. Giả sử mô hình của bạn sinh NaN vì một lỗi thật, chẳng hạn chia cho 0 hoặc lấy log của số âm. Bạn sẽ thấy một quá trình huấn luyện tưởng như bình thường, nhưng thực chất nó bỏ một tỉ lệ lớn số bước. Đáng ghi lại số bước bị bỏ như một chỉ báo riêng.

Weight decay áp nhầm chỗ là lỗi phổ biến và tốn kém: các tham số γ, β của normalization và các bias không nên bị suy giảm, vì chúng không phải “độ phức tạp” theo nghĩa mà weight decay muốn phạt, và co chúng về 0 làm hỏng chính phép chuẩn hoá. Phần lớn recipe tốt loại các nhóm này ra khỏi decay một cách tường minh.

Cuối cùng, những con số trong mục này thuộc ba mức tin cậy khác nhau và nên được đối xử khác nhau. Hệ số $1/(1 - \mu)$, tỉ lệ 3,2 của bước Adam đầu tiên, và các ngưỡng của fp16 là suy dẫn hoặc tra cứu, nên chắc chắn. Kết quả linear scaling tới batch 8192 là một phép đo đã công bố trên một cấu hình cụ thể, không phải một định luật. Còn ngưỡng 10^{-3} cho tỉ lệ cập nhật trên trọng số chỉ là kinh nghiệm thực hành, được truyền lại rộng rãi. Tôi chưa thấy nghiên cứu hệ thống nào xác định lại nó cho các kiến trúc hiện đại. Hãy dùng nó như một cờ báo động, đừng dùng như mục tiêu để tối ưu.

CẠM BÃY

`torch.optim.Adam(..., weight_decay= $\lambda$ )` *không* phải AdamW: nó cộng $\lambda\theta$ vào gradient rồi cho cả cục đi qua mẫu số thích nghi, đúng cái lỗi mà AdamW sinh ra để sửa. Muốn weight decay tách rời thì phải dùng AdamW. Đây là lỗi mà rất nhiều code cũ vẫn mắc, và nó đặc biệt khó phát hiện. Hậu quả không phải một exception mà là điểm số thấp hơn vài phần trăm. Mức đó vừa đủ nhỏ để bạn đổ cho kiến trúc, vừa đủ lớn để mất một vị trí trên bảng xếp hạng.

CẦN NHỚ

Optimizer chỉ định hình lại bước đi; nó không cứu được một learning rate sai. Thứ tự đầu tư công sức: learning rate và lịch của nó trước, batch size cùng quy tắc tỉ lệ sau, chọn

optimizer cuối cùng. Bảy cái tên trong mạch không phải bảy lựa chọn, mà là bảy lần vá liên tiếp trên cùng một vấn đề. [T2]

H. Kết nối

Mục này giả định gradient đã lành mạnh, tức giả định toàn bộ C/12/02-lam-cho-training-chay-duoc. Nó chuyển tiếp sang C/12/04-tong-quat-hoa-va-chan-doan. Ở đó, nhiều gradient của quy tắc tỉ lệ tuyến tính được nhìn lại như một công cụ chính quy hoá. Cũng ở đó, bảng kế toán bộ nhớ của trạng thái Adam trở thành lý do tồn tại của ZeRO. Phần dài dòng của fp16 và bf16 là tiền đề trực tiếp cho D/20-toi-uu-va-nen-mo-hinh (lượng tử hoá), nơi cùng một câu hỏi được đặt cho suy luận thay vì cho huấn luyện, và cho B/11-gioi-han-tinh-toan về phía băng thông. Nếu bạn muốn thấy các lựa chọn trong mục này ảnh hưởng thế nào tới kết luận của một paper, E/24-thi-nghiem-va-ablation là chương nói rằng phần lớn “cải tiến kiến trúc” hoá ra là cải tiến lịch học.

■ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Momentum sửa được gì mà SGD không làm được, và nó tạo ra vấn đề mới nào buộc Nesterov phải ra đời?
2. AdaGrad và RMSProp khác nhau đúng một phép toán. Phép toán đó là gì, và vì sao khác biệt nhỏ ấy lại là ranh giới giữa “dùng được cho mạng sâu” và “không dùng được”?
3. Vì sao AdamW khác Adam cộng L2, và bạn kỳ vọng khác biệt thể hiện ở đại lượng đo được nào?
4. Vì sao warmup cần thiết với Adam hơn là với SGD, nếu trả lời bằng tính chất của bộ ước lượng $\hat{v}$?
5. Bạn tăng batch size gấp 8 và nhân learning rate lên 8; loss huấn luyện tốt hơn nhưng accuracy kiểm thử tụt 1 phần trăm — hai cách giải thích là gì, và phép đo nào phân biệt chúng?
6. Vì sao huấn luyện ở bf16 vẫn cần giữ trọng số fp32, dù bf16 có dải động bằng fp32?
7. Đường cong học giảm nhanh rồi phẳng ở mức cao — ba nguyên nhân khả dĩ là gì, và ba phép đo nào phân biệt chúng?

Nguồn

- [1] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [2] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.
- [3] I. Loshchilov and F. Hutter, “SGDR: Stochastic Gradient Descent with Warm Restarts,” in *ICLR*, 2017.
- [4] P. Goyal, P. Dollár, R. Girshick, et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.
- [5] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [6] S. L. Smith, P.-J. Kindermans, C. Ying, et al., “Don’t Decay the Learning Rate, Increase the Batch Size,” in *ICLR*, 2018.
- [7] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.